SYSTEM ARCHITECTURE & EXPLANATION

Real-Time Distributed Task-Management System Using Java & Python

Challenge Response Document

Candidate	Fardeen
Country	Pakistan
Document	System Architecture & Explanation
Related deliverables	Source code, DB schema, API docs, test reports, deployment guide (see companion files)

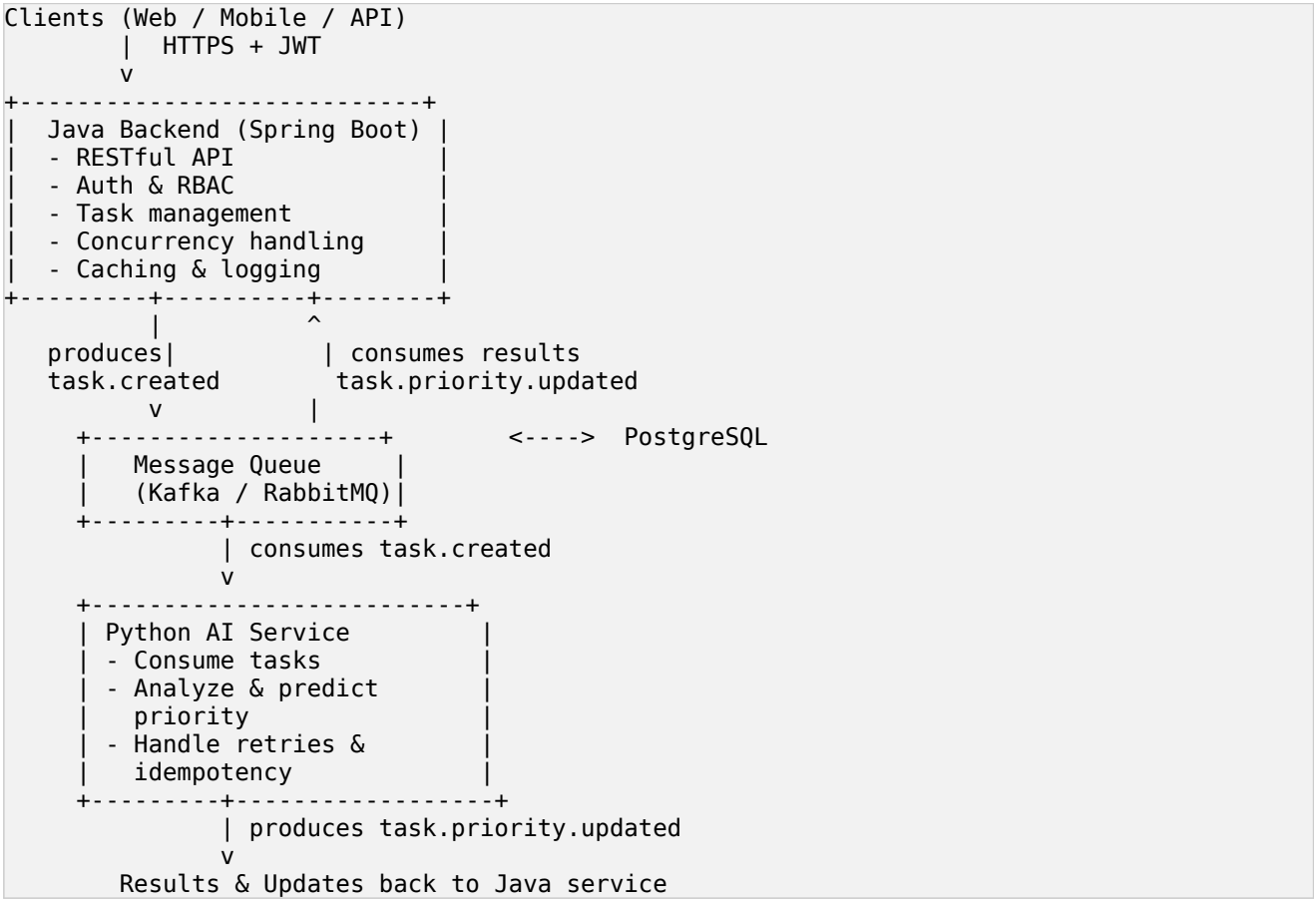
Table of Contents

1. Challenge Summary

Build a real-time distributed task-management system using both Java and Python. Users can create, assign, and track tasks in real time. Java powers the backend API and concurrent task processing; Python provides an AI-powered service that analyzes tasks and predicts their priority. The two services communicate only through a message queue, giving the system durability, retry-safety, and the ability to scale each side independently.

2. High-Level Architecture

The suggested architecture from the challenge brief is implemented as follows:



Data stores: PostgreSQL is the system of record; Redis provides caching, distributed locks, and rate limiting.

3. Requirement-by-Requirement Response

3.1 Java handles the backend API and concurrent task processing

Spring Boot exposes the REST API for tasks, users, and assignments. Concurrency is handled at three levels:

- Database level: optimistic locking via a version column (JPA @Version) on the tasks table.
- Application level: Redis-backed distributed locks / SELECT … FOR UPDATE SKIP LOCKED for multi-step or high-contention operations such as claiming a task.
- Thread pool: a bounded ThreadPoolTaskExecutor handles asynchronous work (Kafka publishing, notifications) off the request thread.

3.2 Python handles an AI/data-processing service

A FastAPI service runs an asynchronous Kafka consumer. On each task.created event it extracts features (due-date proximity, urgency keywords, historical completion time) and classifies the task into LOW / MEDIUM / HIGH / CRITICAL priority, publishing the result back to task.priority.updated.

3.3 Communication only through a message queue

Java and Python never call each other synchronously. All communication happens over Kafka topics:

- task.created — Java to Python
- task.priority.updated — Python to Java
- task.created.DLQ / task.priority.updated.DLQ — dead-letter topics for poison messages

3.4 Authentication, RBAC, transactions, caching, logging

- Auth: JWT access + refresh tokens; passwords hashed with bcrypt; refresh tokens rotated and stored hashed.
- RBAC: ADMIN / MANAGER / MEMBER roles enforced via Spring Security method annotations, plus task-level ownership checks.
- Transactions: @Transactional around multi-table writes; a transactional outbox pattern guarantees the Kafka publish is never lost even if the process crashes right after commit.
- Caching: Redis cache-aside pattern for hot reads (task lists, task detail) with short TTL and explicit invalidation on write.
- Logging: structured JSON logs carrying a correlationId propagated from the HTTP request through Kafka headers into the Python service, enabling end-to-end tracing.

3.5 Supporting 10,000+ concurrent users without losing or duplicating tasks

- Stateless Java instances behind a load balancer; JWT holds session state, not the server.
- HikariCP connection pooling; PostgreSQL read replicas for read-heavy endpoints.
- Kafka partitioned by taskId so ordering is preserved per task while throughput scales with partitions/consumers.
- No lost tasks: durable DB commit before acknowledgment, plus Kafka acks=all with replication factor >= 3.
- No duplicate tasks: client Idempotency-Key header enforced with a unique constraint, and idempotent Kafka consumers on the Python side.

3.6 Automatic retry without duplicates if the Python service crashes

- Idempotent consumption: each event carries a unique eventId; a processed_events table is checked before processing, so redelivery is a no-op if already handled.
- Manual offset commit: the Kafka offset is only committed after the result is durably written and the event marked processed.
- Retry with backoff: transient failures retry with exponential backoff before landing in a DLQ for inspection.
- Transactional outbox on the Java side ensures the original event is never lost even if Java crashes right after the DB commit.

3.7 Database schema, race conditions, and deadlocks

Full DDL is provided in the companion database schema document. Core tables: users, roles, tasks, task_assignees, task_priority_predictions, processed_events (idempotency), outbox_events (transactional outbox), audit_log.

Preventing race conditions

- Optimistic locking (version column) on tasks / task_assignees; a losing writer receives HTTP 409 and retries.
- SELECT … FOR UPDATE SKIP LOCKED for "claim this task" style operations so concurrent claimants never block each other.

Preventing deadlocks

- Row locks are always acquired in a consistent order (e.g., sorted by task_id) when a transaction touches multiple tasks.
- Transactions stay short — no external/network calls while holding a DB lock; Kafka publishing happens after commit via the outbox relay.
- lock_timeout and statement_timeout are set so a stuck transaction fails fast and retries instead of holding locks indefinitely.

3.8 Automated unit, integration, and load tests

- Unit: JUnit5 + Mockito for Java service logic; pytest for Python feature extraction and priority prediction.
- Integration: Testcontainers (real Postgres + Kafka) verifying the outbox to Kafka to consumer round trip, including a test that replaying the same event does not create duplicate results.
- Load: k6 script ramping to 10,000+ virtual users against task creation/read endpoints, asserting p95/p99 latency and error-rate thresholds, and checking that retried requests never create duplicate tasks.

3.9 Deployment and monitoring failures in production

- Each service is containerized independently (separate Dockerfiles) and deployed as separate Kubernetes Deployments.
- CI/CD (GitHub Actions) runs the full test suite, builds images, and performs a rolling deployment gated by readiness/liveness probes.
- Monitoring: Prometheus + Grafana track latency, Kafka consumer lag, DB pool utilization, and inference time; Alertmanager pages on lag growth, elevated error rate, or DB saturation.
- Centralized logs (ELK/OpenSearch) and distributed tracing (OpenTelemetry) correlated by the shared correlationId.
- PostgreSQL runs with a standby replica and automated failover; Kafka topics are replicated across brokers/availability zones.

3.10 Bonus: independent scaling

Because Java and Python communicate only through Kafka, each scales independently: the Java backend scales horizontally on request rate/CPU, while the Python service scales its consumer pods on Kafka consumer lag (via a KEDA ScaledObject) — Kafka automatically rebalances partitions across new pods.

4. Deliverables Checklist

Deliverable	Location

Source code (Java & Python services)	source-code.zip — java-backend/ and python-service/
Database schema & design document	database-schema-and-design.pdf / schema.sql
API documentation	api-documentation.pdf
Test cases & reports (Unit, Integration, Load)	test-cases-and-reports.pdf + tests/ folder
Deployment guide	deployment-guide.pdf
System architecture & explanation (PDF)	this document
Screenshots or demo video	not applicable — no live deployed instance for this exercise

5. Evaluation Criteria Cross-Reference

Criteria	Addressed in
System Design & Architecture	Sections 2, 3
Code Quality & Best Practices	source-code.zip
Scalability & Performance	Sections 3.5, 3.10
Reliability & Fault Tolerance	Sections 3.6, 3.9
Security Implementation	Section 3.4
Testing Coverage	Section 3.8, test-cases-and-reports.pdf
Documentation & Deployment	this document, deployment-guide.pdf